

KU Leuven
Faculty of Engineering Science
Master of Engineering: Computer Science

Ontwikkeling van een live race-engineeringdashboard voor endurancewedstrijden

Jeff Mathieu

Opleidingsonderdeel:	Industriële Stage: Computerwetenschappen
Stageplaats:	MotorSport Training Center
Dagelijkse begeleider:	Jan Wouters
Stageperiode:	22 juni – 31 juli 2026 (zes weken)
Academiejaar:	2026–2027

2 juli 2026

Abstract

Tijdens deze industriële stage van zes weken werd voor MotorSport Training Center een lokale desktopapplicatie ontwikkeld die live timingdata omzet in bruikbare informatie voor een race-engineer. De bestaande timingwebsites tonen de officiële rangschikking en actuele tijden, maar bieden niet alle teamgerichte vergelijkingen, voorspellingen en strategische hulpmiddelen die tijdens een endurancedstrijd nodig zijn. De ontwikkelde Electron-applicatie leest daarom periodiek de timingpagina, normaliseert data van GetRaceResults en RIS Timing, bewaart de racehistoriek lokaal en berekent analyses buiten de gebruikersinterface.

Het resultaat is een dashboard voor maximaal drie gelijktijdig gevolgde wagens, met afzonderlijke modi voor race, kwalificatie en vrije training. Het systeem berekent onder andere ronde- en sectorgemiddelden, geldige beste tijden, voorspelde rondetijden, marges tegenover configureerbare referentietijden, vergelijkingen tussen piloten en klassewagens, inhaalindicaties en een pitstopplanning. Neutralisaties, pitgerelateerde rondes en onvolledige data worden expliciet behandeld. Een afzonderlijk grafiekvenster maakt langere trends zichtbaar. De data wordt per sessie opgeslagen in leesbare JSON-, JSONL- en CSV-bestanden, waardoor een sessie na een onderbreking kan worden hervat.

De ontwikkeling verliep iteratief en in nauw overleg met de opdrachtgever. Praktijktests en screenshots brachten verschillende subtiele fouten aan het licht, onder meer bij rondeachterstanden, intervalvelden, pitstopprojecties en wisselende timingproviders. Deze fouten leidden tot modulaire domeinfuncties en een uitgebreide automatische testsuite. Builds voor Windows en macOS worden via GitHub Actions gemaakt en als release gepubliceerd. De definitieve validatie tijdens de geplande test in Spa is op het moment van schrijven nog niet uitgevoerd en wordt daarom in dit verslag duidelijk als toekomstig werk aangeduid.

Trefwoorden: motorsport, live timing, Electron, race-engineering, rondetijdanalyse, pitstopstrategie, softwaretesten.

Administratieve gegevens

Naam student	Jeff Mathieu
Opleiding	Master in de ingenieurswetenschappen: Computerwetenschappen
Stagebedrijf	MotorSport Training Center
Dagelijkse stagebegeleider	Jan Wouters
Contact begeleider	info@motorsport-trainingcenter.be
Stageperiode	22 juni – 31 juli 2026, zes weken
Opleidingsonderdeel	Industriële Stage: Computerwetenschappen
KU Leuven-coördinator	Prof. Tom Van Cutsem

Inhoudsopgave

Abstract	i
Administratieve gegevens	ii
1 Inleiding en context	1
1.1 Motorsport als beslissingsomgeving	1
1.2 Stageplaats en opdracht	1
1.3 Doelstellingen	2
1.4 Afbakening	2
1.5 Leeswijzer	3
2 Requirements en ontwikkelproces	4
2.1 Iteratief overleg met de klant	4
2.2 Evolutie van de requirements	4
2.3 Functionele vereisten	5
2.4 Niet-functionele vereisten	6
2.5 Planning over zes weken	6
3 Technische architectuur	7
3.1 Electron als desktopplatform	7
3.2 Proces- en modulegrenzen	7
3.3 Vensterarchitectuur	8
3.4 Datastroom	8
3.5 Instellingen en configuratie	9
3.6 Bewuste technische beperkingen	9

4	Afweging van alternatieve oplossingen	10
4.1	Desktopapplicatie tegenover webapplicatie	10
4.2	DOM-extractie tegenover een officiële API	10
4.3	Bestanden tegenover SQLite	11
4.4	Volledig incrementele tegenover herberekende statistiek	11
4.5	Eenvoudige voorspelling tegenover machine learning	11
4.6	Automatische tegenover handmatige modusdetectie	12
4.7	Iteratieve tegenover vooraf vastgelegde ontwikkeling	12
5	Dataverwerking en lokale opslag	13
5.1	Uniforme parser	13
5.2	Detectie van voltooide ronden	13
5.3	Vlagstatus en pitfasen	14
5.4	Bestandsformaat	14
5.5	Incrementele statistiek	14
5.6	Sessiemappen en herstel	15
6	Analyse, voorspelling en strategie	16
6.1	Geldige data als basisregel	16
6.2	Ronde- en pilootstatistieken	16
6.3	Voorspelling van de actuele ronde	17
6.4	Referentietijden	17
6.5	Gevechten binnen de klasse	18
6.6	Pitstopplanner	18
6.7	Full Course Yellow	18
7	Gebruikersinterface en visualisatie	20
7.1	Ontwerp vanuit een operationele schets	20
7.2	Race, practice en qualifying	20
7.3	Meerdere gevolgde wagens	21
7.4	Grafiekvenster	21
7.5	Thema en visuele status	21

7.6	Setupvensters	22
8	Verificatie, simulatie en oplevering	23
8.1	Teststrategie	23
8.2	Mockdata en historische race	23
8.3	Gevonden fouten door realistische scenario's	24
8.4	Coverage en continue integratie	24
8.5	Distributie en updates	25
8.6	Beperkingen van de huidige verificatie	25
9	Evaluatie en persoonlijke reflectie	26
9.1	Evaluatie van het stageplan	26
9.2	Zelfstandig werken	26
9.3	Communicatie en requirementsanalyse	27
9.4	Software-engineeringvaardigheden	27
9.5	Projectmatig werken	27
9.6	Wat beter kon	27
9.7	Meerwaarde	28
10	Relatie met de opleiding	29
10.1	Softwarearchitectuur	29
10.2	Algoritmen en datastructuren	29
10.3	Data-analyse en modellering	30
10.4	Softwaretesten	30
10.5	Human-computer interaction	30
10.6	Communicatie en professionele vaardigheden	30
10.7	Ontbrekende of verder te ontwikkelen kennis	31
11	Validatie tijdens de test in Spa	32
11.1	Doel van de test	32
11.2	Vorbereiding	32
11.3	Te valideren scenario's	32
11.4	Meetplan	33

11.5 Gebruikersfeedback	33
11.6 Resultaat en besluit	34
12 Conclusie	35
A Stageplan	37
B Competenties die ik verwachtte te verbeteren	41

Hoofdstuk 1

Inleiding en context

1.1 Motorsport als beslissingsomgeving

Een endurancewedstrijd is niet alleen een snelheidswedstrijd. Een team moet gedurende meerdere uren de prestaties van de wagen, verschillende piloten, concurrenten in dezelfde klasse, neutralisaties en verplichte pitstops opvolgen. Beslissingen worden genomen terwijl de wedstrijd blijft doorgaan. Informatie heeft daardoor alleen operationele waarde wanneer ze actueel, correct en snel interpreteerbaar is.

De publieke timingpagina is de primaire gegevensbron. Ze toont onder meer positie, klassepositie, wagen- en pilootinformatie, aantal ronden, laatste en beste rondetijd, sectortijden en verschillen tussen wagens. Deze tabel is ontworpen voor een algemeen publiek en voor het wedstrijdverloop als geheel. Een race-engineer stelt echter andere vragen: hoe verhoudt de huidige piloot zich tot zijn teamgenoot, hoe evolueert het tempo van de klasseleider, waar komt de wagen vermoedelijk uit na een pitstop, en nadert de voorspelde rondetijd een opgelegde referentietijd? Het voortdurend handmatig combineren van deze gegevens verhoogt de cognitieve belasting en de kans op rekenfouten.

1.2 Stageplaats en opdracht

De stage werd uitgevoerd voor MotorSport Training Center. De opdrachtgever bracht de motorsportkennis en de operationele verwachtingen aan; de technische analyse en implementatie gebeurden grotendeels zelfstandig. Hierdoor had de opdracht zowel het karakter van softwareontwikkeling als van technische consultancy. Een praktische vraag van de klant moest telkens worden omgezet in ondubbelzinnige gegevens, regels, algoritmes en schermcomponenten.

Het concrete doel was een desktopdashboard dat naast de officiële timingpagina kan draaien. De applicatie moest live data volgen, relevante historiek lokaal bewaren en de informatie specifiek voor het eigen team presenteren. De software moest bruikbaar zijn op Windows tijdens races, maar werd tijdens de stage hoofdzakelijk op macOS ontwikkeld. Platformonafhanke-

lijkheid, eenvoudige installatie en herstel na een onderbreking waren daarom vanaf het begin belangrijke aandachtspunten.

1.3 Doelstellingen

De uiteindelijke opdracht bestond uit de volgende hoofddoelen:

- live timingdata van meerdere websites betrouwbaar lezen en naar één intern formaat vertalen;
- een historiek van rondes, sectortijden en sessiecontext lokaal opslaan;
- maximaal drie wagens volgen zonder de timingwebsite drie keer te bevragen;
- analyses berekenen voor race, kwalificatie en vrije training;
- actuele rondevoorspellingen en configureerbare referentietijden tonen;
- een pitstopvenster en een projectie van de positie na een pitstop aanbieden;
- neutralisaties, pitstops, outlaps en onvolledige data correct behandelen;
- de logica modulair en automatisch testbaar maken;
- installaties en updates voor macOS en Windows via GitHub Releases ondersteunen.

1.4 Afbakening

De applicatie vervangt de officiële timing niet en neemt geen autonome strategische beslissingen. Ze gebruikt uitsluitend informatie die op de timingpagina beschikbaar is of vooraf door de gebruiker wordt geconfigureerd. Brandstofverbruik, bandentemperaturen, telemetrie uit de wagen en weersvoorspellingen maken geen deel uit van de huidige versie. De voorspellingen zijn beslissingsondersteuning en moeten altijd samen met de actuele wedstrijdcontext worden geïnterpreteerd.

Aanvankelijk werd ook gedacht aan replayfunctionaliteit en een uitgebreider machine-learningmodel. Tijdens de iteraties bleek dat betrouwbare live dataverwerking, uitlegbare berekeningen en racebestendige foutafhandeling meer waarde hadden binnen de beschikbare zes weken. Replayfunctionaliteit werd daarom uit de productieapp verwijderd. Voor tests werd wel een lokale simulator gebouwd op basis van historische wedstrijddata uit Assen. De rondevoorspelling gebruikt bewust een transparant model met recente sectorgemiddelden in plaats van een moeilijk uitlegbaar model.

1.5 Leeswijzer

Hoofdstuk 2 beschrijft het iteratieve requirementsproces. Hoofdstuk 3 behandelt de software-architectuur. Hoofdstukken 5 en 6 bespreken de dataverwerking en domeinlogica. Hoofdstuk 7 behandelt de interface. Daarna volgen verificatie en oplevering, de kritische reflectie en een afzonderlijk hoofdstuk met de nog uit te voeren validatie in Spa.

Hoofdstuk 2

Requirements en ontwikkelproces

2.1 Iteratief overleg met de klant

De requirements lagen bij de start niet volledig vast. De opdrachtgever wist welke informatie tijdens een race nodig was, maar de precieze voorstelling en rekenregels werden duidelijker door werkende versies te bekijken. Daarom werd gedurende de zes weken veelvuldig overlegd. Een typische iteratie bestond uit een vraag of schets van de klant, een eerste implementatie, een test met live of historische data en een bespreking van onverwacht gedrag.

Deze aanpak was noodzakelijk omdat ogenschijnlijk eenvoudige velden een domeinspecifieke betekenis hebben. Zo betekent *DIFF* bij GetRaceResults het tijdsverschil met de vorige wagen in de algemene rangschikking, terwijl *INT* bij RIS Timing een interval kan zijn dat in seconden of in ronden wordt uitgedrukt. Een kolom met de titel *IN* bleek dan weer niet voor interval te staan, maar voor de ronde waarin de beste tijd werd gereden. Zulke verschillen kunnen alleen veilig worden verwerkt wanneer zowel de website als realistische wedstrijdsituaties worden onderzocht.

2.2 Evolutie van de requirements

De eerste dashboardversie bevatte een algemene sessiekaart, een klassentabel en grafieken. Feedback leidde tot een volledig hertekend scherm volgens een schets van de opdrachtgever. De informatie werd opgesplitst in grote tijdvakken, compacte sectorvakken, vergelijkingskolommen en een pitstopbalk. Later kwamen daar een donkere modus, een apart grafiekvenster en meerdere gelijktijdige dashboards bij.

Ook de achterliggende regels evolueerden. De oorspronkelijke referentietijdlogica werd eerst verwijderd omdat de betekenis van de regel nog niet voldoende duidelijk was. Nadat de gewenste werking was verduidelijkt, werd ze opnieuw toegevoegd als configureerbare lap- en sectorreferentie met kleurstatussen. De pitstopprojectie begon als een eenvoudige aftrek van de pitstoptijd. Tests toonden vervolgens dat rekening moest worden gehouden met alle

tussenliggende wagens, ook uit andere klassen, met rondeachterstanden, providerafhankelijke intervallen en FCY-omstandigheden.

De opslagstrategie veranderde eveneens. Eerst werd automatisch een nieuwe map per start gemaakt. Dat was handig, maar onveilig wanneer de applicatie per ongeluk werd gesloten: een herstart zou een nieuwe sessie beginnen. De uiteindelijke keuze is dat de gebruiker zelf één map per sessie selecteert. Bij een herstart wordt bestaande JSONL-historiek geladen en worden alleen nieuwe rondes toegevoegd.

2.3 Functionele vereisten

De gerealiseerde functionele vereisten kunnen als volgt worden samengevat:

- F1. De gebruiker configureert timing-URL, sessiemodus, opslagmap en maximaal drie auto-nummers.
- F2. De applicatie bevraagt de timingpagina om de vijf seconden en verdeelt dezelfde data over alle dashboards.
- F3. De parser ondersteunt GetRaceResults en RIS Timing en bewaart de bronprovider.
- F4. Voltooide rondes worden ont dubbeld en persistent opgeslagen.
- F5. Race-, practice- en qualifyingmodus tonen aangepaste analyses.
- F6. Rondes onder FCY, Safety Car of andere neutralisatie blijven opgeslagen maar worden uit representatieve tempoanalyses geweerd.
- F7. Inlaps en outlaps blijven opgeslagen maar tellen niet mee voor tempovergelijkingen.
- F8. Groene sectoren uit een gedeeltelijk geneutraliseerde ronde kunnen afzonderlijk geldig blijven, terwijl de volledige ronde ongeldig is voor het rondetijdgemiddelde.
- F9. De voorspelde rondetijd gebruikt actuele sectoren en recente sectordata van de huidige piloot.
- F10. De gebruiker kan lap- en sectorreferenties aanpassen en ziet de veiligheidsmarge via kleuren en delta's.
- F11. De pitstopplanner houdt rekening met openingsvensters, cooldown, verplichte stops en een instelbare pitstoptijd.
- F12. Grafieken tonen piloot-, sector- en klassevergelijkingen en ondersteunen zoomen bij lange reeksen.
- F13. De app kan na een crash of sluiting verdergaan vanuit dezelfde sessiemap.

2.4 Niet-functionele vereisten

Tijdens een race wegen betrouwbaarheid en begrijpelijkheid zwaarder dan theoretische complexiteit. De berekeningen moesten daarom deterministisch, uitlegbaar en onafhankelijk van de renderer zijn. De renderer krijgt kant-en-klare viewmodellen en formatteert of berekent geen racebeslissingen zelf. Elke belangrijke domeinfunctie moest met synthetische en historische data testbaar zijn.

Verder moest de applicatie langdurig kunnen draaien zonder onbegrensde geheugengroei of overmatige opslag. De pollingfrequentie van vijf seconden levert tijdens 24 uur 17.280 meetmomenten op, maar alleen gewijzigde snapshots en voltooide rondes worden als relevante historiek bewaard. Het gebruik van één collector voor maximaal drie dashboards voorkomt onnodige netwerkbelasting.

2.5 Planning over zes weken

De uitvoering verliep niet als zes strikt gescheiden fasen. In de eerste periode lag de nadruk op parser, live collector en opslag. Daarna volgden dashboardanalyse, tests en de nieuwe UI. Pitstoplogica, voorspelling, sessiemodi en grafieken werden in opeenvolgende iteraties toegevoegd. In de laatste fase kwamen providededgecases, FCY-projecties, distributie, automatische updates en herstel na herstart aan bod.

Deze iteratieve volgorde week af van een klassiek vooraf vastgelegd stageplan, maar paste beter bij de beschikbaarheid van live demo's en klantfeedback. Bugs die alleen in een bijna volledige race zichtbaar werden, hadden een grotere impact op de prioriteiten dan oorspronkelijk geplande uitbreidingen.

Hoofdstuk 3

Technische architectuur

3.1 Electron als desktopplatform

De applicatie is gebouwd met Electron en JavaScript. Electron combineert een Chromium-renderer met een Node.js-mainproces. Deze keuze maakte het mogelijk om een webgebaseerde interface te bouwen en tegelijk lokale bestanden, native dialoogvensters en meerdere desktopvensters te beheren. Dezelfde codebasis kan voor macOS en Windows worden verpakt.

Het mainproces in `src/main/main.js` beheert de levenscyclus van de applicatie, instellingen, vensters, polling, opslag en IPC-handlers. Het verborgen livevenster laadt de echte timingwebsite. Een extractiescript wordt in die pagina uitgevoerd om tabelkoppen, cellen en sessie-informatie uit de gerenderde DOM te lezen. Hierdoor kan de applicatie ook dynamische websites verwerken die niet als eenvoudige statische HTML beschikbaar zijn.

3.2 Proces- en modulegrenzen

De code is bewust in drie zones verdeeld:

- `src/main`: Electron-specifieke infrastructuur, vensters, updates en bestands-I/O;
- `src/shared`: pure domeinfuncties voor parsing, analyse, voorspelling, pitstops en view-modellen;
- `src/renderer`: HTML, CSS en code die reeds berekende data op het scherm zet.

De preloadlaag vormt de gecontroleerde brug tussen renderer en mainproces. `contextIsolation` staat aan en `nodeIntegration` uit, zodat de renderer niet rechtstreeks toegang krijgt tot Node.js. Alleen expliciet aangeboden functies, zoals instellingen ophalen, collectie starten of een grafiekvenster openen, zijn beschikbaar.

De scheiding tussen renderer en domeinlogica was een belangrijke verbetering tijdens de stage. Vroege versies bevatten berekeningen in de UI-code. Daardoor was het moeilijk om te bewijzen

dat een waarde correct was en konden verschillende schermen dezelfde berekening anders uitvoeren. In de huidige architectuur leveren modules zoals `lapAnalytics.js`, `classBattle.js`, `lapPrediction.js` en `pitstopPlanner.js` volledige resultaten. De renderer beperkt zich tot selectie, presentatie en gebruikersinteractie.

3.3 Vensterarchitectuur

Er is één hoofdvenster voor de primaire wagen. Wanneer de gebruiker via de setup een tweede of derde wagen toevoegt, opent voor elke extra wagen een dashboardvenster met dezelfde renderer en een andere wagenparameter. Alle vensters ontvangen dezelfde collectorstatus. De timingwebsite wordt dus slechts eenmaal geladen en bevraagd.

Grafieken worden in afzonderlijke vensters geopend. Ook zij gebruiken de reeds opgeslagen lap history en starten geen eigen collector. Deze aanpak houdt het hoofdscherm compact en maakt het mogelijk om analysevensters alleen te openen wanneer ze nodig zijn.

Een verborgen `BrowserWindow` blijft tijdens collectie actief. Wanneer de gebruiker de timingpagina ter debugging toont en daarna sluit, wordt dit venster normaal verborgen in plaats van vernietigd. De algemene applicatielifecycle vormt daarop een uitzondering: bij een echte Quit worden polling en close-to-hide uitgeschakeld. Deze aparte lifecyclemodule lost een macOS-probleem op waarbij de app anders in de Dock actief bleef en alleen met Force Quit kon worden beëindigd.

3.4 Datastroom

Elke poll doorloopt dezelfde keten:

1. Het extractiescript leest tabellen en sessievelden uit de livepagina.
2. De parser herkent bruikbare headers en zet elke rij om naar canonieke velden.
3. De storage-normalizer voegt provider- en sessiecontext toe.
4. Nieuwe voltooide ronden worden via een stabiele identiteit ontdubbeld.
5. De lap history en actuele rijen worden aan de analysemodules doorgegeven.
6. Voor elke gevolgde wagen worden dashboardanalyse, voorspelling en pitstopplan berekend.
7. De volledige collectorstatus wordt opgeslagen en via IPC naar alle vensters gestuurd.

De UI en de berekeningen worden dus gevoed door dezelfde consistente snapshot. Dit voorkomt dat een grafiek bijvoorbeeld met gegevens van een andere poll rekent dan het hoofdscherm.

3.5 Instellingen en configuratie

Instellingen worden als JSON in de Electron user-datafolder bewaard. Ze omvatten timing-URL, gevolgde wagens, modus, thema, opslagmap, referentietijden en pitstopregels. Referentietijden zijn afzonderlijk per sessiemodus opgeslagen, zodat een qualifyingreferentie een racereferentie niet overschrijft. Circuitprofielen voor Spa, Zolder en Assen bevatten standaardwaarden voor de relevante afstand en FCY-snelheid. In de pitstopsetup kunnen deze waarden per evenement worden overschreven wanneer het reglement afwijkt.

3.6 Bewuste technische beperkingen

De timingwebsite blijft een externe afhankelijkheid. Een wijziging van DOM-structuur of kolomnamen kan parseraanpassingen vereisen. Daarom bewaart de applicatie parserdiagnostiek en blijft de livepagina zichtbaar via een debugactie. Ook worden onbekende of onbetrouwbare waarden niet stilzwijgend als nul geïnterpreteerd: analysefuncties retourneren waar mogelijk een expliciete onbeschikbare toestand.

Hoofdstuk 4

Afweging van alternatieve oplossingen

4.1 Desktopapplicatie tegenover webapplicatie

Een centrale webapplicatie was een mogelijk alternatief voor Electron. Een webserver zou updates voor alle gebruikers onmiddellijk beschikbaar maken en meerdere clients kunnen bedienen. Daartegenover staan extra operationele afhankelijkheden: hosting, authenticatie, netwerkconfiguratie en bereikbaarheid van de server. De timingbron is zelf al afhankelijk van internet. Door verwerking en opslag lokaal te houden wordt ten minste de reeds ontvangen historiek niet van een tweede externe dienst afhankelijk.

Een gewone lokale webserver met browserinterface had eveneens gekund. Electron voegt pakketgrootte en geheugengebruik toe, maar vereenvoudigt native bestandskeuze, vensterbeheer, installatie en distributie. Vooral het openen van afzonderlijke dashboards voor extra wagens en grafieken past goed bij een desktopapplicatie. Voor dit prototype woog die ontwikkelsnelheid zwaarder dan de grotere runtime.

Een volledig native Windowsapplicatie, bijvoorbeeld in C#, zou een kleinere en nauwer geïntegreerde productieapp kunnen opleveren. De ontwikkeling gebeurde echter op macOS en het resultaat moest ook daar testbaar blijven. Electron maakte één gedeelde implementatie mogelijk. Wanneer de applicatie later zeer breed wordt uitgerold en resourcegebruik problematisch blijkt, kan een native frontend opnieuw worden onderzocht terwijl de pure domeinregels als specificatie behouden blijven.

4.2 DOM-extractie tegenover een officiële API

Een officiële JSON- of WebSocket-API zou betrouwbaarder zijn dan DOM-extractie. Velden zouden een gedocumenteerd type en vaste semantiek kunnen hebben. Voor de gebruikte publieke timingpagina's was zo'n uniforme beschikbare API niet het uitgangspunt. De applicatie

leest daarom de gerenderde tabel die ook de gebruiker ziet.

Deze oplossing is pragmatisch maar kwetsbaar voor HTML-wijzigingen. De impact wordt beperkt door headercanonicalisatie, providerdetectie en parserdiagnostiek. Wanneer in de toekomst toegang tot een officiële feed beschikbaar wordt, kan een nieuwe collectoradapter worden toegevoegd. De downstreammodules hoeven dan niet te veranderen zolang dezelfde genormaliseerde rijen worden geproduceerd.

4.3 Bestanden tegenover SQLite

SQLite was aanvankelijk de voor de hand liggende keuze. Het biedt transacties, indexes en krachtige queries zonder afzonderlijke server. Voor de uiteindelijke schaal bleken bestanden echter voordelen te hebben. Een engineer kan een CSV direct openen, JSONL kan append-only worden geschreven en een beschadigde laatste regel tast niet noodzakelijk de volledige historiek aan. Ook export en debugging zijn eenvoudiger.

Het nadeel is dat relaties en constraints in applicatiecode worden bewaakt. Queries over meerdere grote evenementen zijn minder efficiënt. Wanneer later een centraal archief met vele races en interactieve historische analyses nodig is, wordt SQLite of PostgreSQL relevanter. De huidige storage schema-module biedt daarvoor al een uniforme representatie; migratie hoeft niet vanuit willekeurige rendererdata te gebeuren.

4.4 Volledig incrementele tegenover herberekende statistiek

Een gemiddelde kan efficiënt worden bijgewerkt met een teller en som. Voor een lange race lijkt dit aantrekkelijk. In dit project zijn ronden echter niet altijd definitief geldig op het eerste moment dat ze verschijnen. Pitstatus, sectorvlaggen of correcties kunnen de selectie beïnvloeden. Een gecachet gemiddelde zou dan teruggedraaid moeten worden met exact dezelfde eerdere context.

Daarom wordt de lap history als bron gebruikt en worden statistieken opnieuw uit de gefilterde relevante ronden berekend. Met enkele duizenden records en pure JavaScriptfuncties blijft dit snel genoeg. De laatst berekende samenvatting wordt wel opgeslagen voor inspectie. Pas wanneer metingen aantonen dat herberekenen een werkelijk prestatieprobleem vormt, is een incrementele cache met invalidatieregels verantwoord.

4.5 Eenvoudige voorspelling tegenover machine learning

Een regressiemodel, random forest of neurale netwerk zou meerdere features kunnen combineren: piloot, bandenslijtage, brandstofniveau, verkeer, temperatuur en weersomstandigheden. De timingpagina levert veel van die inputs niet. De beschikbare trainingsdata is bovendien beperkt en bevat veranderende circuits en omstandigheden. Een complex model zou daardoor

snel overfitten en moeilijk uitlegbaar zijn.

Het sectorgemiddeldenmodel heeft duidelijke beperkingen, maar iedere component kan op het dashboard worden verklaard. Het model reageert op actuele sectoren en gebruikt alleen recente data van dezelfde piloot. Een toekomstige uitbreiding kan deze baseline vergelijken met exponentiële weging of regressie. Een complexer model is pas zinvol wanneer het op historische sessies aantoonbaar een lagere fout heeft en zijn onzekerheid operationeel kan communiceren.

4.6 Automatische tegenover handmatige modusdetectie

De sessienaam bevat vaak woorden als Race, Qualifying of Practice. Automatische detectie lijkt dus eenvoudig. Toch verschillen talen, afkortingen en providerlabels, en een organisator kan een afwijkende naam gebruiken. Een fout gedetecteerde modus kan pitinformatie verbergen of verkeerde vergelijkingen tonen. Daarom kiest de gebruiker de modus expliciet. De extra handeling is klein en het gedrag blijft voorspelbaar.

4.7 Iteratieve tegenover vooraf vastgelegde ontwikkeling

Een watervalaanpak met een volledige specificatie vóór implementatie zou minder herwerking beloven. In deze opdracht bestond veel kennis echter impliciet bij de opdrachtgever. Pas een zichtbaar dashboard maakte duidelijk welke informatie ontbrak. Een iteratieve aanpak met korte feedbacklussen was daardoor geschikter.

Dit betekende niet dat elke nieuwe wens onmiddellijk zonder afweging werd toegevoegd. De architectuur werd gaandeweg strenger: gedeelde logica moest naar pure modules, nieuwe edgecases kregen regressietests en providerafhankelijke aannames moesten expliciet worden benoemd. Zo bleef het prototype ondanks veranderende requirements onderhoudbaar.

Hoofdstuk 5

Dataverwerking en lokale opslag

5.1 Uniforme parser

De timingproviders gebruiken verschillende labels en representaties. De parser begint daarom met het opschonen van witruimte en het canoniseren van headers. Varianten worden gekoppeld aan interne sleutels voor positie, startnummer, klasse, PIC, team, piloot, aantal ronden, gap, interval, sectoren, laatste tijd, beste tijd en pitinformatie. Een headercontrole voorkomt dat een willekeurige tabel op de pagina als timingtabel wordt gebruikt.

Tijdswaarden worden zo vroeg mogelijk naar milliseconden omgezet. De tekst `2:05.073` wordt intern bijvoorbeeld 125.073 milliseconden. Formatteringsfuncties zetten dit pas bij presentatie opnieuw om. Dit maakt optellen, middelen, vergelijken en testen eenduidig.

Elke genormaliseerde rij bevat ook de gedetecteerde bronprovider. Dat is noodzakelijk omdat een identiek ogend veld een andere semantiek kan hebben. `GetRaceResults` levert een bruikbare *DIFF* tussen opeenvolgende algemene posities. Om de afstand tussen twee klassegagens te bepalen worden alle tussenliggende *DIFF*-waarden opgeteld, ook die van wagens uit een andere klasse. RIS Timing gebruikt *INT*; wanneer die waarde een ronde-eenheid bevat, wordt de rondeachterstand conservatief omgerekend met een representatieve gemiddelde rondetijd. De code bevat daarnaast fallbacks voor onvolledige feeds.

5.2 Detectie van voltooide ronden

De live tabel bevat zowel de laatst voltooide rondetijd als sectoren van de ronde die bezig is. Bij elke poll wordt de laatste bekende rij per wagen bewaard. Wanneer de combinatie van wagen, ronde en tijd een nieuwe voltooide ronde aanduidt, wordt een laprecord gemaakt. De sectorwaarden van de vorige snapshot vormen dan het beste bewijs voor de sectoren van die voltooide ronde.

Een lap identity voorkomt duplicaten wanneer dezelfde live rij gedurende meerdere polls zichtbaar blijft. Bij een herstart worden identities uit de bestaande JSONL-historiek opnieuw in

een set geladen. Nieuwe data kan daardoor veilig aan dezelfde sessie worden toegevoegd zonder oude ronden opnieuw te schrijven.

5.3 Vlagstatus en pitfasen

Voor elke sector wordt de vlagstatus vastgelegd op het moment dat de sector beschikbaar wordt. Dit detail is nodig voor het scenario waarin sector 1 groen wordt gereden en tijdens sector 2 FCY begint. De volledige ronde is dan niet representatief voor een rondetijdgemiddelde, maar sector 1 kan nog geldig zijn voor het gemiddelde en de beste waarde van sector 1.

De analyselaag annoteert ook pitfasen. Veranderingen in pitteller en pitstatus helpen inlaps en outlaps identificeren. Deze ronden blijven deel van de historische werkelijkheid, maar worden niet gebruikt voor representatief tempo. Dit voorkomt dat een trage pitronde het gemiddelde van een piloot kunstmatig verslechtert.

5.4 Bestandsformaat

Er werd uiteindelijk niet voor SQLite gekozen. Voor de schaal van één race en de behoefte aan inspecteerbaarheid bleken leesbare bestanden eenvoudiger en voldoende. De belangrijkste bestanden zijn:

- `lap_history.jsonl`: append-only historiek met één JSON-object per ronde;
- `lap_history.csv`: tabelvorm voor manuele inspectie of spreadsheetanalyse;
- `latest_live_rows.json` en `.csv`: laatste volledige timingssnapshot;
- `session_metadata.json`: sessiecontext;
- `parser_debug.json`: gedetecteerde headers, voorbeelden en waarschuwingen;
- `analytics_summary.json`: laatst berekende statistieken;
- aparte prediction- en pitstopplanbestanden per gevolgde wagen.

JSONL is geschikt voor rondes omdat een nieuwe ronde kan worden toegevoegd zonder het volledige bestand te herschrijven. Tegelijk blijft elke regel leesbaar en herstelbaar. Samenvattingsbestanden worden wel overschreven: ze vertegenwoordigen de actuele afgeleide toestand en kunnen indien nodig opnieuw uit de lap history worden berekend.

5.5 Incrementele statistiek

De applicatie bewaart naast ruwe historiek ook de laatste analysesamenvatting. Dit maakt inspectie eenvoudig, maar de lap history blijft de gezaghebbende bron. Gemiddelden worden

bij een update opnieuw opgebouwd uit de relevante gefilterde rondes. Voor de verwachte omvang van maximaal enkele duizenden rondes is dit computationeel klein en betrouwbaarder dan complexe incrementele correcties. Wanneer een ronde later als pit- of neutralisatieronde wordt herkend, zou een uitsluitend incrementeel gemiddelde immers ook moeten worden teruggedraaid.

5.6 Sessiemappen en herstel

De gebruiker selecteert voor elke echte sessie een eigen map. De applicatie maakt niet bij iedere druk op Start automatisch een nieuwe timestampmap. Daardoor kan dezelfde map na een crash of foutieve sluiting opnieuw worden geselecteerd. Bij het starten laadt de applicatie bestaande lap history en pitstatus, bouwt de duplicate guard op en vervolgt de collectie.

Deze keuze vraagt discipline: voor een werkelijk nieuwe race, kwalificatie of training moet een nieuwe lege map worden gekozen. In ruil daarvoor is hervatten expliciet en voorspelbaar. De opslagcalculator in de testtools schat de benodigde ruimte op basis van aantal wagens, race-uren en pollinginterval. Zelfs bij een 24-uurswedstrijd blijft de bestandsgrootte beheersbaar, omdat snapshots compact zijn en afgeronde rondes veel minder frequent voorkomen dan polls.

Hoofdstuk 6

Analyse, voorspelling en strategie

6.1 Geldige data als basisregel

Vrijwel elke analyse begint met dezelfde vraag: is deze ronde representatief? `lapAnalytics.js` centraliseert deze beslissing. Een ronde onder Safety Car, Full Course Yellow, Code 60 of een andere neutralisatie wordt opgeslagen, maar niet gebruikt voor gemiddelden, best-driververgelijkingen of tempografieken. Hetzelfde geldt voor pitgerelateerde in- en outlaps. Sectoren worden afzonderlijk beoordeeld, zodat een groene sector uit een later geneutraliseerde ronde niet verloren gaat.

Deze centrale filtering voorkomt dat verschillende functies uiteenlopende definities van een geldige ronde gebruiken. Grafieken, gemiddelden en vergelijkingen steunen daardoor op dezelfde regels. De tests bevatten expliciete scenario's met ontbrekende tijden, gedeeltelijke neutralisatie, pitfasen en ongeldige waarden.

6.2 Ronde- en pilootstatistieken

Voor een reeks geldige ronden worden laatste tijd, beste tijd, gemiddelde tijd, sectorgemiddelden en beste sectoren bepaald. Pilootstatistieken worden per combinatie van wagen en piloot opgebouwd. De beste piloot van het eigen team is niet hard gecodeerd: wanneer de actuele piloot na zijn stint sneller blijkt, kan hij de vorige referentiepiloot vervangen.

In racemodus vergelijkt het dashboard onder meer:

- beste teamtijd met de laatste ronde van de huidige piloot;
- beste teamtijd met de beste ronde van de huidige piloot;
- gemiddeld tempo van de beste piloot met dat van de huidige piloot;
- tempo van de beste wagen in de klasse met de eigen actuele stint;
- tempo van een vrij gekozen klassewagen met de eigen actuele stint.

In qualifyingmodus zijn gemiddelden minder relevant. Daar gebruikt de app vooral beste en laatste geldige ronde van teamgenoten en concurrenten. Practice legt de nadruk op referenties en consistente vergelijking zonder pitstopstrategie.

6.3 Voorspelling van de actuele ronde

De rondevoorspelling is bewust eenvoudig en uitlegbaar. Voor de huidige piloot worden per sector maximaal de laatste tien geldige sectortijden geselecteerd. Na sector 1 is de voorspelling:

$$\hat{T}_{lap} = T_{S1,actueel} + \bar{T}_{S2,recent} + \bar{T}_{S3,recent}.$$

Na sector 2 worden beide actuele sectoren gebruikt:

$$\hat{T}_{lap} = T_{S1,actueel} + T_{S2,actueel} + \bar{T}_{S3,recent}.$$

Na sector 3 wordt de voorspelling niet vervangen door de echte rondetijd. Zo blijft zichtbaar wat na sector 2 voorspeld was. Tussen de finish en de nieuwe sector 1 toont het dashboard de afwijking tussen voorspelling en gerealiseerde ronde. Dit geeft onmiddellijke feedback over de kwaliteit van het model.

Voor een nieuwe piloot is na zijn eerste geldige volledige ronde voldoende basisdata beschikbaar om vanaf sector 1 van zijn tweede ronde een eerste voorspelling te maken. Wanneer een piloot eerder al een stint reed, wordt zijn eigen historische sectordata opnieuw gebruikt. Een schuivend venster zorgt ervoor dat oude droge data bij regen geleidelijk plaatsmaakt voor recente natte data. De voorspelling convergeert daardoor zonder de oude kennis abrupt te verwijderen.

6.4 Referentietijden

De gebruiker kan een referentierondetijd en drie sectorreferenties ingeven via editknoppen in de betrokken vakken. De waarden worden per sessiemodus opgeslagen. `normReference.js` berekent de delta en deelt de toestand in als veilig, nabij of kritisch. De grenswaarden zijn centraal configureerbaar.

Het vak voor predicted lap time krijgt een duidelijke groene, oranje of rode achtergrond. De ideal time is de som van de beste sectoren van de wagen, niet enkel van de huidige piloot, en krijgt eveneens een status. Bij elke sectorreferentie worden twee marges getoond: die van de laatste sector en die van de beste sector. Hierdoor ziet de engineer zowel de actuele uitvoering als het theoretische risico.

6.5 Gevechten binnen de klasse

Een tijdsverschil tussen twee klassewagens mag niet worden afgeleid door alleen hun zichtbare klassepositie te vergelijken. Er kunnen wagens uit andere klassen tussen staan. Voor `GetRaceResults` telt `classBattle.js` daarom de opeenvolgende DIFF-waarden in de algemene rangschikking op. Voor RIS Timing worden providerafhankelijke intervallen verwerkt en kunnen ronde-eenheden met representatieve rondetijden worden benaderd.

Wanneer wagens in dezelfde ronde zitten, combineert de applicatie de actuele gap met het verschil tussen recente gemiddelden. Zo ontstaat een indicatie van het aantal ronden of de tijd tot een mogelijke inhaalactie. Wanneer de rondeafstand of de data onvoldoende betrouwbaar is, wordt geen schijnpreciese voorspelling getoond.

6.6 Pitstopplanner

De pitstopplanner gebruikt configureerbare regels voor totale raceduur, verboden eerste en laatste 25 minuten, cooldown na een geldige stop, aantal verplichte stops en geschatte pitduur. De balk toont rode en groene zones en bewaart reeds verstreken rode cooldownsegmenten. Alleen stops die binnen een geldig groen venster plaatsvinden, tellen mee voor het verplichte aantal.

De planner berekent ook het laatste veilige moment waarop nog voldoende verplichte stops met cooldown kunnen plaatsvinden. Hierdoor wordt zichtbaar wanneer een ogenschijnlijk toegestane late stop de resterende strategie onmogelijk zou maken. De pitduur blijft tijdens de race aanpasbaar, omdat een bandenwissel en een pilotenwissel verschillende tijden kunnen vragen.

Voor de projectie na een pitstop worden de relatieve verschillen in de algemene rangschikking opgebouwd totdat de geschatte pit loss is overschreden. Vervolgens wordt bepaald tussen welke klassewagens de eigen wagen zou uitkomen. Dit lost het probleem op waarbij een wagen uit een andere klasse tussen twee relevante klassewagens stond. Rondeverschillen worden niet als seconden gelezen; ze worden met beschikbare gemiddelde rondetijden geïnterpreteerd.

6.7 Full Course Yellow

Onder FCY rijdt het veld met een opgelegde snelheid en kan de relatieve pit loss kleiner zijn. Circuitprofielen bevatten daarom de afstand die een niet-pittende wagen tussen pit entry en pit exit over het gewone circuit aflegt en de FCY-snelheid. De tijd op dat traject is:

$$T_{track,FCY} = \frac{d_{track}}{v_{FCY}/3,6}.$$

De geschatte netto pit loss vergelijkt deze trajecttijd met de ingestelde pitduur. Omdat

timinggaps vlak na de overgang naar FCY nog kunnen veranderen, bewaart de app opeenvolgende signatures en markeert de projectie eerst als voorlopig. Zodra een nieuwe passage en stabiele gaps zijn waargenomen, kan de projectie betrouwbaarder worden gebruikt. Afstanden en snelheden uit een circuitprofiel blijven overschrijfbaar voor afwijkende wedstrijdregels.

Deze berekening blijft een model. Ze kent geen lokale opstopping, pit-entryvertraging, Safety Car-wave-by of onverwachte serviceproblemen. De interface presenteert haar daarom als projectie en niet als garantie.

Hoofdstuk 7

Gebruikersinterface en visualisatie

7.1 Ontwerp vanuit een operationele schets

De definitieve interface is gebaseerd op een door de opdrachtgever aangeleverde schets. De nadruk ligt op informatiedichtheid en snelle herkenning, niet op een marketingachtige presentatie. Bovenaan staat een compacte infobalk met sessie, resterende tijd, wagen, huidige piloot, klassepositie, status en acties. Grote vakken tonen laatste en beste rondetijd. Daaronder staan actuele, referentie- en beste sectoren. Rechts krijgen voorspelling, ideal time en de geselecteerde vergelijkingswagen een vaste plaats.

De onderste vergelijkingsmatrix groepeerde waarden per betekenis. Elke kolom bevat twee bronwaarden en een delta. Positieve en negatieve uitkomsten krijgen een groene of rode rand volgens hun praktische betekenis. Delta's worden compact zonder overbodige minuten weergegeven, terwijl grote tijdverschillen in de pitstopprojectie wel als minuten en seconden verschijnen.

7.2 Race, practice en qualifying

De modus wordt handmatig in Setup gekozen. Automatische detectie uit de sessienaam werd bewust niet gebruikt, omdat providers sessies anders benoemen en een fout gekozen modus operationeel verwarrend kan zijn.

Race toont het pitstopvenster, recente tempoverschillen en catchindicaties. Practice verbergt racegebonden pitinformatie en focust op referenties, sectoren en vergelijkingen. Qualifying gebruikt beste en laatste tijden in plaats van langetermijngemiddelden. Dezelfde interface blijft herkenbaar, maar de betekenisvolle data verandert per sessietype.

7.3 Meerdere gevolgde wagens

In Setup staat naast het primaire autonummer een plusknop. Hiermee kunnen maximaal twee extra wagens worden toegevoegd. Elke wagen krijgt een eigen dashboardvenster, maar gebruikt dezelfde live snapshot en opslag. Dit is belangrijk voor een team dat tijdens een 24-uurswedstrijd meerdere wagens inzet: de website wordt niet extra belast en alle schermen zijn onderling consistent.

7.4 Grafiekvenster

Via een compact grafiekicoon in de bovenbalk opent een afzonderlijk analysevenster. Vier grafieken kunnen gelijktijdig worden getoond en elk vak heeft een dropdown om het grafiektype te wijzigen. De standaardselectie omvat:

- geldige rondetijden per piloot;
- beste, gemiddelde en recente prestaties per piloot;
- gemiddelde en beste sectoren per piloot;
- werkelijke rondetijden van alle wagens in dezelfde klasse.

Bij rondetijden per piloot wordt de geldige-rondeteller gebruikt. De derde geldige ronde van piloot A en de derde geldige ronde van piloot B staan daardoor op dezelfde x-positie, ook wanneer hun stints op verschillende momenten plaatsvonden. FCY-, Safety Car-, in- en outlaps laten geen lege plaats achter. Voor grafieken met veel ronden is een zoombereik voorzien, zodat een deel van de wedstrijd kan worden onderzocht zonder de volledige reeks onleesbaar samen te drukken.

7.5 Thema en visuele status

De applicatie ondersteunt een licht en donker thema via een zon- of maanicoon. Kleuren zijn gegroepeerd in CSS-variabelen, zodat het palet later zonder componentwijzigingen kan worden aangepast. De donkere variant gebruikt geen zuiver zwart; panelen blijven visueel van elkaar onderscheiden en rood, oranje en groen behouden voldoende contrast.

Kleur heeft een functionele betekenis. De predictionkaart gebruikt niet alleen een gekleurde rand maar een lichtere volledige achtergrond, zodat een kritische referentietijd direct opvalt. De pitstopcontainer krijgt groen wanneer stoppen momenteel toegestaan is en rood wanneer het venster gesloten is. Tekst blijft daarnaast altijd aanwezig, zodat de toestand niet uitsluitend van kleurwaarneming afhangt.

7.6 Setupvensters

De algemene setup bevat timing-URL, modus, gevolgde wagens en sessiemap. Een afzonderlijke pitstopsetup bevat vaste race-informatie: totale raceduur, verplichte stops, circuit/pitconfiguratie, reguliere trajectafstand en FCY-snelheid. Deze waarden worden voor de race ingesteld en tijdens actieve collectie vergrendeld. Alleen de verwachte pitduur blijft rechtstreeks op het dashboard wijzigbaar.

Referentietijden worden dichterbij hun gebruik aangepast: elk reference-vak heeft een klein editicoon. Hierdoor hoeft de gebruiker niet naar Setup om tijdens een sessie een regelwaarde bij te stellen.

Hoofdstuk 8

Verificatie, simulatie en oplevering

8.1 Teststrategie

Een groen testresultaat is alleen waardevol wanneer de verwachte uitkomst onafhankelijk van de implementatie is bepaald. Daarom gebruiken de tests expliciete scenario's met handmatig berekende tijden, grenzen en negatieve gevallen. Ze controleren niet enkel dat een functie een waarde retourneert, maar ook welke rondes worden geselecteerd, welke eenheid wordt gebruikt en wanneer een resultaat bewust onbekend moet blijven.

De testsuite wordt uitgevoerd met Node.js en `assert`. `tests/run-all.js` ontdekt automatisch alle bestanden met suffix `.test.js`. Daardoor volstaat een nieuw testbestand om het in de volledige suite op te nemen. De suite bevat momenteel tests voor parser, storage schema, sessieherstel, analytics, voorspelling, referentietijden, klassegevechten, pitstopplanning, circuitprofielen, grafiekdata, sessiemodi, rendererviewmodellen, updater en applicatielifecycle.

8.2 Mockdata en historische race

Voor statistische functies werd een centrale mockhistoriek gemaakt met meerdere piloten, stints, sectoren en ongeldige rondes. Hiermee worden gemiddelde rondetijden, snelste rondes, sectorgemiddelden, beste sectoren en vergelijkingen met klassewagens gecontroleerd. Edgecases omvatten lege reeksen, nullwaarden, onbekende piloten, negatieve of onrealistische tijden, gedeeltelijke sectorinformatie en wissels tussen piloten.

Daarnaast werd historische data van een Belcar-race in Assen gebruikt. PDF-bestanden met startgrid, klassering, rondetijden en sectoranalyse werden omgezet naar een compacte dataset voor de Club Challenge-klasse. Een lokale HTTP-simulator presenteert deze data als een timingpagina en kan de race versneld laten verlopen, bijvoorbeeld één ronde per vijf of tien seconden. De simulator heeft eigen tests; afzonderlijke analytics-tests gebruiken dezelfde historische data om de berekeningsfuncties te controleren.

Deze twee soorten tests hebben een ander doel. Simulatortests bewijzen dat de nepwebsite

correct evolueert. Analytics-tests bewijzen dat domeinfuncties de verwachte uitkomst geven. De simulator alleen kan geen fout in een berekening detecteren wanneer dezelfde fout ook in de verwachte simulatorlogica zou zitten.

8.3 Gevonden fouten door realistische scenario's

Een aantal belangrijke problemen werd alleen zichtbaar met realistische data:

- Bij een nieuwe sessiemap verscheen aanvankelijk ronde 60, hoewel de live demo verder stond. De oorzaak was dat wijzigingen in de laatste rondetijd als nieuwe lokale rondes werden geteld zonder de echte rondecontext correct te gebruiken.
- Een prediction van ongeveer 1:35 voor een wagen met ronden rond 2:05 wees op fout geïnterpreteerde sectorwaarden en onvoldoende samplevalidatie.
- Een klassewagen leek slechts twaalf seconden achter te liggen, terwijl een tussenliggende wagen één ronde achterstand had. De oplossing gebruikt de algemene rangschikking en alle tussenliggende intervallen.
- Een pitstopprojectie meldde duizenden seconden voorsprong doordat een rondeveld als tijd werd geïnterpreteerd. Provider- en eenheidsdetectie werden aangescherpt.
- Safety Car- en FCY-ronden maakten grafieken onleesbaar en gemiddelden fout. De gedeelde eligibilityfuncties sluiten ze nu overal consistent uit.
- Een verborgen livevenster blokkeerde op macOS het normaal afsluiten van de app. Een geteste lifecyclemodule maakt bij Quit alle verborgen vensters echt afsluitbaar.

Deze voorbeelden tonen waarom alleen happy-path-tests onvoldoende zijn. Veel fouten ontstaan precies wanneer een veld syntactisch geldig is maar semantisch iets anders betekent.

8.4 Coverage en continue integratie

Naast `npm test` is een coveragerun beschikbaar via `npm run test:coverage`. Statement-, function- en branchcoverage worden gebruikt als aanwijzing voor vergeten paden. Vooral branchcoverage is nuttig voor fallbacklogica, nullwaarden, providerkeuzes en gesloten/open pitvensters. Een hoog percentage bewijst echter niet dat de verwachtingen correct zijn; daarom blijft scenarioselectie belangrijker dan het getal alleen.

GitHub Actions voert bij elke push en pull request zowel tests als coverage uit op Ubuntu met Node.js. Branch protection kan deze workflow als verplichte statuscheck gebruiken voordat een pull request naar main wordt gemerged.

8.5 Distributie en updates

`electron-builder` maakt een NSIS-installer voor Windows en DMG/ZIP-bestanden voor macOS. Een tag zoals `v1.1.1` activeert de releaseworkflow. Mac- en Windows-builds worden parallel gemaakt, waarna één GitHub Release met alle platformbestanden wordt gepubliceerd.

De geïnstalleerde applicatie controleert bij het starten via `electron-updater` op een nieuwere GitHub Release. Developmentruns via `npm start` doen dit bewust niet. De app en installers zijn momenteel niet digitaal ondertekend of genotariseerd. Windows SmartScreen en macOS Gatekeeper kunnen daarom waarschuwingen tonen. De workflow bevat reeds plaatsen voor toekomstige signingsecrets.

8.6 Beperkingen van de huidige verificatie

Automatische tests modelleren bekende scenario's. Ze vervangen geen langdurige test met echte timingproviders, netwerkonderbrekingen en de operationele gebruiker. De Assen-data is waardevol, maar vertegenwoordigt niet alle circuits, reglementen en timingimplementaties. De geplande test in Spa moet daarom de volledige keten valideren: installatie, polling, opslag, analyses, pitstopprojectie, meerdere schermen en interpretatie door de opdrachtgever.

Hoofdstuk 9

Evaluatie en persoonlijke reflectie

9.1 Evaluatie van het stageplan

Het oorspronkelijke plan legde terecht de nadruk op dataverzameling, lokale opslag, visualisatie en voorspelling. De precieze technische invulling veranderde echter sterk. SQLite werd vervangen door transparante JSONL-, JSON- en CSV-bestanden. Replay werd uit de eindapp verwijderd en vervangen door een testgerichte simulator. Een complex AI/ML-traject maakte plaats voor een uitlegbare sectorvoorspelling. Tegelijk groeide de race-engineeringfunctionaliteit met meerdere sessiemodi, pitstopplanning, providerondersteuning, meerdere dashboards, grafieken en automatische releases.

Die verschuiving was verantwoord. Binnen zes weken was het belangrijker om de volledige live keten betrouwbaar te maken dan om een complex model bovenop onzekere data te bouwen. De moeilijkste problemen zaten niet in de rekenkundige formules, maar in de betekenis en kwaliteit van timingvelden. Zonder correcte providerinterpretatie zou een geavanceerd model alleen preciezere foutieve resultaten produceren.

9.2 Zelfstandig werken

De technische uitvoering gebeurde grotendeels zelfstandig. Dit vereiste dat problemen systematisch werden verkleind: eerst de ruwe snapshot inspecteren, dan de parseroutput, vervolgens het opgeslagen laprecord en ten slotte de analyse. Door functies uit de renderer te halen konden fouten lokaal met kleine inputs worden gereproduceerd.

Zelfstandigheid betekende ook beslissen wanneer een vraag technisch kon worden opgelost en wanneer domeinfeedback nodig was. De betekenis van een referentietijdregel of geldige pitstop kan niet uit code worden afgeleid. In zulke gevallen was overleg met de opdrachtgever noodzakelijk voordat verdere implementatie zinvol was.

9.3 Communicatie en requirementsanalyse

Het frequente overleg met de klant was een centraal deel van de stage. Schermafbeeldingen en concrete scenario's bleken effectiever dan abstracte technische uitleg. Een vraag als “waar staan we na een pitstop van 75 seconden?” maakte onmiddellijk zichtbaar welke data en aannames ontbraken. De implementatie kon vervolgens worden uitgelegd in termen van opeenvolgende verschillen, rondeachterstanden en klasseposities.

Ik leerde requirements niet als vaste lijst te behandelen. De opdrachtgever kon pas na gebruik beoordelen of een veld groot genoeg was, of een kleur voldoende opviel en of een vergelijking tijdens een race werkelijk nuttig was. Tegelijk moest ik bewaken dat elke wijziging een duidelijk betekenis hield en niet alleen visueel plausibel werd.

9.4 Software-engineeringvaardigheden

De stage bracht verschillende onderdelen uit de opleiding samen: modulariteit, datastructuren, foutafhandeling, testontwerp, user-interfaceontwerp en distributie. Vooral separation of concerns werd praktisch belangrijk. Pure shared modules bleken niet alleen beter testbaar, maar maakten ook meerdere dashboards en grafiekvensters mogelijk zonder logica te kopiëren.

Het project maakte ook het verschil duidelijk tussen syntax en semantiek. Een parser kan een tekst perfect lezen en toch een fout resultaat produceren wanneer 5L als vijf seconden wordt behandeld. Betrouwbare software vereist daarom domeinconstraints en tests met betekenisvolle voorbeelden.

9.5 Projectmatig werken

De zes weken dwongen tot prioritering. Nieuwe ideeën, zoals weersafhankelijke strategie, bandenkeuze en een uitgebreid FCY-model, waren relevant maar konden niet allemaal volledig worden uitgewerkt. De gekozen werkwijze was telkens eerst een minimale end-to-endfunctie bouwen, die met realistische data testen en pas daarna verfijnen.

Versiebeheer en releases vormden een nuttige afsluiting van iteraties. Featurebranches werden via pull requests naar main gebracht. Tags genereerden reproduceerbare platformbuilds. Automatische tests verminderden het risico dat een nieuwe pitstop- of parserwijziging oudere functionaliteit brak.

9.6 Wat beter kon

Een formele provider-specificatie aan het begin had sommige interpretatiefouten kunnen voorkomen. In de praktijk waren de feeds echter vooral via demo's en screenshots beschikbaar. Ook had een vroeg centraal model voor “geldige tempo-data” duplicatie kunnen vermijden;

dit inzicht ontstond pas nadat neutralisatieronden op meerdere plaatsen fout werden meegenomen.

De README groeide mee met de ontwikkeling en bevat daardoor op enkele plaatsen historische formuleringen. Voor een volgende fase is het zinvol documentatie per release te controleren tegen de actuele setupflow. Ook digitale signing, notarization en een formeel migratiepad voor opgeslagen data zijn nodig voordat de app breder buiten de huidige testcontext wordt verspreid.

9.7 Meerwaarde

Voor de opdrachtgever levert de stage een werkende basis die ruwe timing vertaalt naar teamgerichte inzichten en verder kan worden uitgebreid. Voor mij lag de grootste meerwaarde in het bouwen van software waarvan een fout tijdens live gebruik directe operationele gevolgen kan hebben. Dat maakte voorzichtigheid, uitlegbaarheid en testen concreter dan in een geïsoleerde academische opdracht.

Hoofdstuk 10

Relatie met de opleiding

10.1 Softwarearchitectuur

De meest directe relatie met de opleiding ligt in softwareontwerp. Het project vereiste duidelijke verantwoordelijkheden tussen infrastructuur, domeinlogica en presentatie. Concepten als encapsulation en separation of concerns kregen een praktisch gevolg: wanneer de renderer zelf een pitstopgap berekende, was die moeilijk afzonderlijk te testen en te hergebruiken. Na modularisatie kon dezelfde berekening het dashboard, opslagbestand en een test voeden.

Ook interfaces tussen componenten waren belangrijk. De parser levert genormaliseerde rijen, analytics ontvangt laprecords en viewmodules leveren schermdata. Deze contracten vermindern de koppeling met één timingprovider of één venster. Het project illustreerde daarmee waarom een architectuur niet alleen een diagram is, maar vooral een verzameling afdwingbare grenzen in de code.

10.2 Algoritmen en datastructuren

Hoewel de datasets relatief klein zijn, komen klassieke datastructuren voortdurend terug. Maps bewaren de laatste rij en pitstatus per wagon. Sets met lap identities voorkomen duplicaten. Gesorteerde reeksen reconstrueren algemene en klassevolgorde. Schuivende vensters selecteren recente geldige ronden.

De gapberekening is een voorbeeld van algoritmisch redeneren op een sequentie. Om het verschil tussen twee klassegagens te kennen, wordt niet alleen naar hun eigen rij gekeken. Alle tussenliggende algemene rijen worden doorlopen en geldige opeenvolgende intervallen worden opgeteld. Edgecases zoals ronde-eenheden, ontbrekende waarden en omgekeerde volgorde maken van een eenvoudige som een domeinalgoritme met precondities.

10.3 Data-analyse en modellering

Gemiddelden, minima en rolling windows zijn eenvoudige statistieken, maar hun kwaliteit hangt af van dataselectie. De stage maakte concreet dat preprocessing vaak belangrijker is dan het model. Een gemiddelde met Safety Car-ronden is rekenkundig correct en inhoudelijk waardeloos voor tempoanalyse.

De rondevoorspelling is een baseline-regressiemodel. De fout kan na iedere ronde direct worden gemeten. Dit sluit aan bij de opleidingsprincipes rond train/testscheiding, geschikte evaluatiemetrieken en het vergelijken met een eenvoudige baseline. De huidige implementatie gebruikt nog geen formele offline modeltraining, maar legt wel de datakwaliteits- en evaluatiebasis voor een latere uitbreiding.

10.4 Softwaretesten

De tests dwongen tot het expliciet formuleren van verwachte domeinregels. Branchcoverage wees op niet-uitgevoerde fallbackpaden, maar handberekende scenario's bepaalden of de code inhoudelijk juist was. Dit onderscheid tussen verificatie van uitvoering en validatie van betekenis is een belangrijk software-engineeringinzicht.

Continue integratie breidde dit uit naar het ontwikkelproces. Een lokale succesvolle test is niet voldoende wanneer een dependency of platformomgeving afwijkt. GitHub Actions installeert vanaf een schone checkout en voert de volledige suite opnieuw uit. Releases worden pas na dezelfde tests gebouwd.

10.5 Human-computer interaction

De raceomgeving stelt specifieke HCI-eisen. Een gebruiker heeft geen tijd om meerdere geneste menu's te doorlopen of lange uitleg te lezen. Informatiehiërarchie, stabiele vakafmetingen en kleurstatussen moesten daarom samen worden ontworpen. Het verschil tussen een dunne rode rand en een volledig lichtrood predictionvak bleek operationeel relevant: de tweede variant trekt sneller aandacht.

Tegelijk mag kleur niet de enige informatiedrager zijn. Labels, delta's en statustekst blijven zichtbaar. De keuze om grafieken in een afzonderlijk venster te plaatsen is eveneens een HCI-afweging: detailanalyse blijft beschikbaar zonder het primaire racescherf permanent te belasten.

10.6 Communicatie en professionele vaardigheden

De opleiding biedt een technische basis, maar deze stage vroeg ook vertaling tussen twee vakgebieden. De opdrachtgever formuleerde wensen vanuit racesituaties, niet als softwaretic-

kets. Ik moest doorvragen naar uitzonderingen, voorbeelden met concrete wagens uitwerken en vervolgens technische beperkingen begrijpelijk uitleggen.

Het project leerde mij ook onzekerheid expliciet te communiceren. Wanneer timingdata onvoldoende betrouwbaar is, is “onbekend” professioneler dan een exact ogend fout getal. Dit geldt zowel voor de interface als voor overleg. Technische geloofwaardigheid ontstaat niet door altijd een antwoord te tonen, maar door duidelijk te maken waarop een antwoord gebaseerd is.

10.7 Ontbrekende of verder te ontwikkelen kennis

Voor een volgende fase zou meer kennis over officiële motorsporttimingprotocollen en race-controlregels nuttig zijn. Ook code signing, notarization en productieobservability kwamen slechts beperkt in de opleiding aan bod maar zijn belangrijk voor distributie van desktopsoftware. Aan de datazijde zijn onzekerheidsintervallen en concept drift relevante onderwerpen wanneer de voorspelling verder wordt uitgebouwd.

De stage toonde daarmee zowel de toepasbaarheid als de grenzen van de academische voorbereiding. De opleiding leverde het gereedschap om een modulair systeem te ontwerpen en problemen analytisch op te lossen. De operationele context leerde welke details bepalen of dat systeem werkelijk bruikbaar is.

Hoofdstuk 11

Validatie tijdens de test in Spa

NOG UIT TE VOEREN: De praktijktest in Spa heeft nog niet plaatsgevonden. Dit hoofdstuk beschrijft het validatieplan, niet de resultaten. Alle meetwaarden, observaties en conclusies moeten na de test worden ingevuld.

11.1 Doel van de test

De test in Spa moet aantonen of de applicatie buiten demo- en historische data bruikbaar blijft gedurende een echte operationele sessie. De nadruk ligt niet alleen op correcte cijfers, maar ook op installatie, stabiliteit, begrijpelijkheid en de snelheid waarmee de race-engineer informatie kan interpreteren.

11.2 Voorbereiding

Voor de sessie moeten timingprovider, URL, gevolgde wagens, sessiemodus en een lege opslagmap worden ingesteld. Voor racegebruik moeten raceduur, verplichte stops, gekozen Spa-pitconfiguratie, reguliere trajectafstand en FCY-snelheid worden gecontroleerd met het actuele reglement. Referentietijden worden pas ingevoerd wanneer de opdrachtgever de juiste waarden bevestigt.

NOG UIT TE VOEREN: Noteer hier de gebruikte appversie, laptop en besturings-systeem, timing-URL, Spa-layout, betrokken autonummers, sessietype, reglementaire pitstopregels en referentietijden.

11.3 Te valideren scenario's

Minstens de volgende punten moeten tijdens of onmiddellijk na de test worden gecontroleerd:

1. Komt iedere voltooide ronde precies eenmaal in de historiek terecht?

2. Komen pilootwissels en actuele sectoren tijdig op het juiste dashboard?
3. Worden inlaps, outlaps en neutralisatieronden correct opgeslagen en uitgesloten?
4. Sluit de predicted lap time na sector 1 en 2 voldoende aan bij de uiteindelijke ronde?
5. Reageren referentiekleuren en delta's op de geconfigureerde regels?
6. Komen verschillen met voorliggende en achterliggende klassewagens overeen met de officiële timing?
7. Is de pitstopprojectie voor green flag en FCY plausibel vóór en na een echte stop?
8. Blijven cooldown, verplichte-stopteller en herstartstatus correct?
9. Blijven maximaal drie dashboards en het grafiekvenster synchroon?
10. Kan de app na sluiten of een gesimuleerde crash met dezelfde map hervatten?

11.4 Meetplan

Voor elke voorspelde ronde worden voorspelling na sector 1, voorspelling na sector 2 en werkelijke rondetijd geregistreerd. Daaruit kunnen absolute fouten worden berekend. Voor pitstops worden voorspelde klassepositie, voorspelde gap en werkelijke toestand na stabilisatie genoteerd. Bij afwijkingen moet ook de vlagstatus en kwaliteit van de timingfeed worden bewaard.

Tabel 11.1: In te vullen meettabel voor rondevoorspellingen in Spa.

Ronde	Piloot	Voorspelling S1	Voorspelling S2	Werkelijk
–	–	–	–	–
–	–	–	–	–

NOG UIT TE VOEREN: Vul na de test gemiddelde absolute fout, grootste afwijking en observaties per weers- of vlagconditie in. Voeg geen geschatte waarden toe wanneer een meting ontbreekt.

11.5 Gebruikersfeedback

Naast technische metingen moet de opdrachtgever beoordelen welke velden daadwerkelijk tijdens de sessie werden bekeken, welke waarschuwingen duidelijk waren en welke informatie te veel plaats innam. Ook moet worden gevraagd of de pitstopprojectie voldoende uitlegbaar is om er een beslissing op te baseren.

NOG UIT TE VOEREN: Voeg hier concrete feedback van de opdrachtgever toe, inclusief voorgestelde wijzigingen en hun prioriteit.

11.6 Resultaat en besluit

NOG UIT TE VOEREN: Schrijf na de Spa-test een feitelijk besluit: welke onderdelen zijn gevalideerd, welke fouten zijn gevonden, welke fixes zijn uitgevoerd en welke risico's blijven open. Tot dan mag dit hoofdstuk niet als bewijs van praktijkvalidatie worden gebruikt.

Hoofdstuk 12

Conclusie

Tijdens deze zesweekse stage werd een eerste operationele versie van het MSTC Race Engineer Dashboard ontwikkeld. De applicatie leest live timingpagina's, normaliseert providerafhankelijke gegevens, bewaart een hervatbare racehistoriek en zet die om in analyses voor race, kwalificatie en vrije training. De interface ondersteunt maximaal drie wagens met één gedeelde collector en biedt daarnaast een afzonderlijk grafiekvenster.

De belangrijkste technische bijdrage is niet één afzonderlijk algoritme, maar de betrouwbare keten van externe timingdata naar uitlegbare beslissingsinformatie. Rondes en sectoren worden met hun context opgeslagen. Neutralisaties en pitfasen worden bewaard zonder de representatieve gemiddelden te vervormen. De rondevoorspelling gebruikt recente sectordata van de juiste piloot. Klasseverschillen en pitstopprojecties houden rekening met algemene racevolgorde, tussenliggende wagens, ronde-eenheden en timingprovider. De renderer presenteert deze resultaten maar berekent ze niet zelf.

Het intensieve overleg met de opdrachtgever was bepalend voor het resultaat. Praktische scenario's brachten fouten aan het licht die met generieke testdata moeilijk zichtbaar waren. Door die scenario's om te zetten naar afzonderlijke domeinfuncties en regressietests groeide de software van een dashboardprototype naar een beter onderhoudbare applicatie. De automatische test- en releaseworkflows ondersteunen verdere ontwikkeling na de stage.

Niet alle ambities zijn definitief afgerond. De voorspellingen blijven eenvoudige, uitlegbare modellen en gebruiken geen telemetrie, banden- of weerdata. De FCY-pitloss is afhankelijk van correcte circuitparameters en stabiele timinggaps. Code signing en notarization ontbreken nog. Het belangrijkste open punt is de praktijktest in Spa. Tot die test is uitgevoerd, kunnen stabiliteit en nauwkeurigheid in een echte raceomgeving niet definitief worden bevestigd.

De stage maakte duidelijk dat race-engineeringsoftware vooral betrouwbaar moet omgaan met onvolledige en contextafhankelijke data. Een plausibel getal is niet noodzakelijk een correct getal. Door onbekende situaties zichtbaar te maken, aannames configureerbaar te houden en regels automatisch te testen, biedt de gerealiseerde applicatie een stevige basis voor verdere validatie en uitbreiding.

Bibliografie

- [1] Electron. *Electron Documentation*. Geraadpleegd in 2026. <https://www.electronjs.org/docs/latest/>
- [2] GetRaceResults. *LiveTiming Circuit Zolder*. Geraadpleegd in 2026. <https://livetiming.getraceresults.com/zolder>
- [3] RIS Timing. *Live timing results*. Geraadpleegd in 2026. <https://results.ris-timing.be/>
- [4] KU Leuven. *Industriële stage: Computerwetenschappen*. Onderwijsaanbod 2025–2026. <https://onderwijsaanbod.kuleuven.be/syllabi/n/H04I9A>
- [5] Faculteit Ingenieurswetenschappen KU Leuven. *Schriftelijk rapporteren*. <https://eng.kuleuven.be/studeren/engineering-essentials/rapporteren/schriftelijk>
- [6] Mathieu, J. *Stageplan: Jeff Mathieu – MotorSport Training Center*. Ongepubliceerd stageplan, 2026. Toegevoegd als bijlage.
- [7] Mathieu, J. *Competenties die ik verwachtte te verbeteren tijdens mijn stage*. Ongepubliceerd document, 2026. Toegevoegd als bijlage.

Bijlage A

Stageplan

Stageplan: Jeff Mathieu <> MotorSport Training Center

Student	Jeff Mathieu
Opleiding	Master Computer Science
Stageperiode	22 juni - 31 juli (6 weken)
Dagelijkse begeleider	Jan Wouters, [FUNCTIE], info@motorsport-trainingcenter.be, +32 475 39 41 16
Fysische locatie	[Locatie]

1. Probleemstelling

Tijdens een endurancewedstrijd moet een race-engineer snel en betrouwbaar beslissingen nemen op basis van live timingdata. De bestaande live timingomgeving van Circuit Zolder toont veel ruwe informatie, zoals rondetijden, sectortijden, posities en andere sessie-informatie. Deze informatie is echter niet specifiek afgestemd om de concrete beslissingen te maken die een team tijdens de race moet nemen. Voor een team is het vooral belangrijk om de prestaties van de eigen wagen en de relevante klasse overzichtelijk te volgen. Een belangrijke regel die teams worden opgelegd is de referentietijdregel. Deze regel is er zodat teams niet sneller kunnen rijden dan de andere auto's in hun klasse. Het is dus belangrijk dat het team een waarschuwing krijgt wanneer deze referentietijdregel in gevaar komt. Door de verwachte rondetijden in te kunnen schatten kunnen er strategische beslissingen gemaakt worden.

Een belangrijk probleem wat waarschijnlijk gaat voorkomen is dat de site met live timingdata tijdelijk kan wegvallen. Een praktisch dashboard mag daarom niet afhankelijk zijn van één browservenster. De applicatie moet dus ontvangen data lokaal bewaren, ook kunnen werken met replay- of simulatiegegevens wanneer er geen race bezig is en na een herstart of onderbreking opnieuw bruikbaar zijn zonder historiek te verliezen.

Daarnaast willen we onderzoeken of historische en live rondetijdgegevens gebruikt kunnen worden om rondetijden van piloten te voorspellen over korte tijdsintervallen. Deze voorspellingen kunnen helpen om tijdsverlies of tijdswinst over bijvoorbeeld 15 minuten, 30 minuten en 1 uur te schatten en zo de potstrategie beter te ondersteunen.

2. Concrete doelstelling

Het doel van de stage is het ontwerpen en implementeren van een lokaal draaiende softwaretool die timingdata verwerkt, analyseert en visualiseert. Het eindresultaat is een werkend prototype dat bruikbaar is in simulatie/replaymodus en live de race kan volgen.

Concreet verwachten we volgende resultaten:

- Een dashboard dat voor de auto rondetijden, sectortijden, gemiddelden, snelste rondes en trends toont
- Een module die referentietijden per sessie configureerbaar maakt en waarschuwingen geeft bij kritieke rondetijden of mogelijke overtredingen
- Een lokale databank waarin timingevents, rondes, sectortijden, correcties en sessiegegevens persistent worden opgeslagen
- Een replay- en simulatiemodus waarmee het systeem getest kan worden zonder actieve race
- Een eerste voorspellend model dat op basis van beschikbare data rondetijden en tijdverlies over 15, 30 en 60 minuten inschat
- Een duidelijke gebruikersinterface waarmee een race-engineer snel de status van de wagen kan zien en hiermee strategie beslissingen kan maken.

3. Activiteiten en opdrachten

Ik zal binnen het team deelnemen aan de volgende activiteiten en opdrachten: live timingdata verwerken, dashboard ontwikkelen, rondetijden analyseren, AI/ML-voorspellingen maken, tijdsverlies inschatten, pitstrategie ondersteunen, resultaten testen en valideren met de dagelijkse begeleider.

Ik kan het systeem testen door de replay-functionaliteit te implementeren en gebruiken. Als het prototype ver genoeg gevorderd is om het tijdens een raceweekend te testen, kan dit vanuit thuis of tijdens de race op het circuit zodat ik tijdens de race kan laten zien hoe het prototype werkt.

4. Technische aanpak

De applicatie wordt opgevat als een local-first dashboard. De gebruikersinterface staat los van de opslag en verwerking van de timingdata. De timingcollector haalt gegevens op uit de bron, normaliseert deze naar een intern formaat en bewaart ze in een lokale database. De analyselaag berekent vervolgens gemiddelden, marges ten opzichte van referentietijden, risico-indicatoren en voorspellingen. De dashboardlaag toont deze resultaten aan de gebruiker.

De implementatie zal bij voorkeur bestaan uit een desktop- of lokale webapplicatie met een moderne frontend, een backendlaag voor dataverwerking en een SQLite-database voor lokale opslag. De software moet ontwikkeld kunnen worden op macOS en later kunnen draaien op een Windows-laptop. Daarom worden platformafhankelijke paden en instellingen vermeden. Indien mogelijk wordt ook een eenvoudige manier voorzien om de applicatie te verpakken of te starten op Windows.

Voor de AI/ML-component wordt gestart met een eenvoudige en uitlegbare baseline, bijvoorbeeld een rolling-average- of regressiemodel op basis van recente rondetijden, sectortijden, piloot, stuntfase, pitstatus en eventuele vlag-of neutralisatieperiodes. Daarna kan onderzocht worden of een uitgebreider model betere voorspellingen oplevert. De nauwkeurigheid wordt geëvalueerd door voorspelde rondetijden en tijdsverlies te vergelijken met historische of geployde sessiedata.

5. Werkplan en timing

Deze stage duurt 6 weken. Onderstaande planning is een inschatting van de activiteiten die ik die week ga aanvangen. In overleg met de begeleider kan deze planning worden aangepast naarmate prioriteiten veranderen doorheen het project.

	Taak	Omschrijving
Week 1	Analyse en specificatie	Exacte vereisten overleggen, beschikbare timing data onderzoeken, datamodellen onderzoeken, referentietijd-regel begrijpen.
Week 2	Dataverwerking en opslag	Locale database ontwerpen, import/replay van historische sessiedata voorzien om testen te kunnen uitvoeren de volgende weken.
Week 3	Dashboard en basisanalyse	Hoofdschermen bouwen voor overzicht, rondetijden, sectortijden, gemiddelden, snelste rondes en waarschuwingen.
Week 4	Robuustheid en simulatie	Replaymodus uitbreiden, onderbrekingen en correcties simuleren, persistentie en herstel na herstart testen.
Week 5	AI/ML-voorspelling	Voorspellingsmodel ontwikkelen voor rondetijden en tijdsverlies, resultaten evalueren en visualiseren.
Week 6	Validatie en afwerking	Dashboard testen met begeleider, feedback verwerken, documentatie schrijven, beperkingen en verdere uitbreidingen beschrijven, eindprototype opleveren.

6. Verwachte deliverables

- Werkend prototype van het timingdashboard met lokale opslag
- Import- en replaymogelijkheid voor historische of gesimuleerde timingdata
- Analysefuncties voor rondetijden, sectortijden, gemiddelden, referentietijdmarges en waarschuwingen
- AI/ML-module voor voorspellingen van rondetijden en tijdsverlies
- Korte technische documentatie om “klant” zelfstandig deze tool te laten gebruiken

7. Stageplaats en begeleiding

Ik al slechts op bepaalde momenten ter plaatse aanwezig zijn, bijvoorbeeld voor overleg, feedbackmomenten of praktische afstemming met de begeleider. Dit aangezien de begeleider zelf vaak op verplaatsing werkt. Het grootste deel van de technische uitwerking zal zelfstandig van thuis uit gebeuren.

De begeleider levert vooral domeinkennis, requirements (meeste vooraf afgesproken, er kunnen natuurlijk altijd requirements veranderen/bijkomen), feedback vanuit een race-engineering perspectief. De communicatie zal voornamelijk via e-mail of telefoon verlopen. Ik zal regelmatig vragen stellen, mijn voortgang bespreken en feedback verwerken om te controleren of het dashboard aansluit bij de praktische doeleinden van het raceteam.

Bijlage B

Competenties die ik verwachtte te verbeteren

Competenties die ik verwacht te verbeteren tijdens mijn stage

1. Zelfstandig werken

Ik zal een groot deel van de analyse, het ontwerp en de implementatie zelfstandig uitvoeren.

2. Samenwerken en afstemmen

Ik leer mijn werk regelmatig afstemmen met de begeleider en feedback verwerken in het project.

3. Communicatie met een niet-IT klant

Ik leer technische keuzes duidelijk uitleggen aan iemand met vooral domeinkennis uit de motorsport.

4. Requirements analyseren

Ik zal noden uit de praktijk vertalen naar concrete functies voor het timingdashboard.

5. Data-analyse en AI/ML

Ik leer timingdata verwerken en gebruiken om rondetijden en mogelijk tijdsverlies te voorspellen.

6. Projectmatig werken

Ik leer een technisch project plannen, testen, bijsturen en opleveren binnen een vaste stageperiode.